



Overview



REMEDIATION REPORT
for
MEGAQUAGGA PUBLISHING



MQP

Version 1.0 (August 10, 2025)

Prepared by **Chibuikwe Ojukwu**

Presented to **MEGAQUAGGA PUBLISHING**

DISCLAIMER: This document contains proprietary and confidential information. 0x2A SECURITY guarantees the confidentiality of the information and any other material that has or will be disclosed during the performance of this service. A mutual non-disclosure agreement of confidential information is assumed between 0x2A SECURITY and MEGAQUAGGA PUBLISHING, limiting the use of confidential information exclusively for purposes associated with properly delivering this service.

EXECUTIVE SUMMARY

During the remediation process, I implemented multiple security measures to stop ongoing attacks targeting the MegaQuagga WordPress site. These steps included configuring a reverse proxy, enabling ModSecurity as a Web Application Firewall (WAF), and securing traffic with SSL offloading. Each stage was verified through network packet analysis to confirm that malicious requests, specifically Remote File Inclusion (RFI) attempts were detected and blocked before any harmful payloads could be delivered or executed.

These remediations addressed critical vulnerabilities that could allow attackers to gain unauthorized access to server files, execute malicious code, and compromise sensitive system data. By blocking RFI attempts and preventing payload execution, we eliminated the immediate threat of server compromise, protected user data, and reduced the risk of future exploitation. As a result, the website is now operating with stronger defenses against common web-based attacks, significantly improving the overall security posture.



Actions Taken

ACTIONS TAKEN

This section outlines the actions taken on which assets to remediate which vulnerabilities.

Vulnerability	Remediation Solution
WordPress is vulnerable to web application attacks	Implemented ModSecurity Web Application Firewall (WAF) in active blocking mode with OWASP CRS ruleset to detect and block malicious requests, including Remote File Inclusion (RFI) attempts.
Unencrypted web traffic	Configured SSL/TLS offloading on the reverse proxy using a valid X.509 certificate to enforce HTTPS for all inbound and outbound WordPress communications.

Remediation Solution #1: ModSecurity WAF Deployment

Step-by-step Implementation:

- Vulnerability Discovery:**
 - Detected Remote File Inclusion (RFI) attempts in PCAP analysis, originating from 192.168.100.20 targeting /wp-admin/admin-post.php with swp_url parameter pointing to a remote payload.
 - Requests were confirmed via Wireshark filters (http.request.uri contains "admin-post.php" && http.request.uri contains "swp_url=") and validated as successful before WAF deployment.
- Asset Targeted:**
 - WordPress application hosted on 10.20.30.220 behind pfSense LAN.
- Remediation Actions:**
 - Installed and enabled **ModSecurity** on the reverse proxy server.
 - Applied the **OWASP Core Rule Set (CRS)** to cover common web exploits, including RFI, SQL injection, and XSS.
 - Switched ModSecurity from **passive detection** to **active prevention** mode to block malicious traffic in real-time.
 - Configured logging to store all blocked request details for forensic review.
- Result:**
 - Subsequent PCAP capture confirmed HTTP/1.1 403 Forbidden responses to RFI attempts, with payload retrieval blocked.
 - No unauthorized remote code execution detected post-deployment.

Remediation Solution #2: SSL/TLS Offloading Configuration

Step-by-step Implementation:

1. Vulnerability Discovery:

- Observed plaintext HTTP traffic in PCAP captures, including sensitive WordPress login and request data.
- Confirmed lack of HTTPS enforcement, leaving communication vulnerable to interception.

2. Asset Targeted:

- Reverse proxy server handling inbound traffic to WordPress server (10.20.30.220).

3. Remediation Actions:

- Installed a valid **SSL/TLS certificate** on the reverse proxy.
- Configured the proxy to handle **SSL/TLS offloading**, decrypting incoming HTTPS requests and forwarding secure traffic internally.
- Enforced **HTTP to HTTPS redirection** for all inbound requests.
- Updated WordPress site settings (Site URL and Home URL) to use https://.

4. Result:

- All web traffic to WordPress is now encrypted in transit.
- PCAP captures confirmed TLS handshake and absence of plaintext credentials or sensitive data.

☒ Evidence of Remediation

EVIDENCE OF REMEDIATION

This section documents evidence of the remediations being implemented. This evidence may take the form of details from packet captures (for example, alterations in traffic flow or packet headers), ModSecurity logs appearing in the Graylog SIEM, or output from command-line tools. For each remediation, we provide evidence of the original state as well as the current altered state.

Reverse proxy

These are alterations to MegaQuagga’s environment associated with implementing the reverse proxy server responsible for implementing our two remediations (WAF & SSL offloading).

Item	Original Vulnerable State	Current Remediated State
/cf/conf/config.xml	Reverse proxy not configured, allowing direct access to WordPress application, exposing it to web application attacks and bypassing any protective filtering.	Configured HAProxy reverse proxy to route all external traffic through a single entry point, enabling filtering and logging before requests reach the WordPress backend.
pfSense-LAN-1.pcap (Packets 399–436)	Packet captures show direct traffic to the backend WordPress server without proxy filtering. RFI request (pkt 399) reaches the backend directly and receives payload execution response (pkt 436).	In pfSense-LAN-2.pcap (Packets 4–62), traffic now flows through reverse proxy IP with altered headers (X-Forwarded-For), confirming proxy intervention before reaching the backend.

Web Application Firewall (WAF)

These are alterations to MegaQuagga's environment associated with implementing the WAF remediation.

Item	Original Vulnerable State	Current Remediated State
/cf/conf/config.xml	ModSecurity WAF disabled, allowing Remote File Inclusion (RFI) attempts to execute and return sensitive file contents (/etc/passwd). Example: pfSense-LAN-2.pcap (pkt 4-62) shows HTTP 200 OK with /etc/passwd returned.	Enabled ModSecurity WAF with OWASP CRS, blocking malicious RFI payloads. Example: pfSense-LAN-3.pcap (pkt 27-29) shows identical RFI attempt receiving HTTP 403 Forbidden.
pfSense-LAN-2.pcap (Packets 4, 58, 62)	RFI attack bypasses filtering, payload from /etc/passwd visible in HTTP response payload.	pfSense-LAN-3.pcap (Packets 27-29) - no payload returned, ModSecurity log confirms rule triggered and request blocked.

SSL Offloading

These are alterations to MegaQuagga's environment associated with implementing the WAF remediation.

Item	Original Vulnerable State	Current Remediated State
/cf/conf/config.xml	WordPress site served entirely over HTTP, leaving traffic unencrypted and vulnerable to interception (credentials, cookies, session tokens exposed).	Installed SSL certificate on reverse proxy and enabled HTTPS redirection, encrypting all inbound connections before forwarding to backend.
pfSense-LAN-3.pcap (Packets 27-29)	All HTTP traffic in cleartext; payloads including credentials and queries visible in packet capture.	pfSense-LAN-4.pcap (Packets 5-29) – traffic now encrypted over TLS; packet payload unreadable in Wireshark.
Browser Test (manual verification)	Browser shows "Not Secure" HTTP connection to site.	Browser shows valid HTTPS lock icon; certificate inspection confirms trusted CA and correct hostname.

Recommendations

RECOMMENDATIONS

This section outlines recommendations for continued monitoring and next steps for future remediations to further strengthen MegaQuagga's security posture.

Recommendation #1: **Implement HTTP Strict Transport Security (HSTS)**

Description: Configure the reverse proxy (HAProxy) to include the Strict-Transport-Security header with a long max-age and preload option. This enforces HTTPS-only connections, even if a user manually enters an HTTP URL or follows an unsecured link, closing the window for SSL stripping and protocol downgrade attacks.

Recommendation #2: **Deploy a Web Application Security Platform (WASP) Layer**

Description: Extend protection beyond ModSecurity by integrating a cloud-based WAF/CDN (e.g., Cloudflare, AWS WAF) to absorb DDoS attempts, block known bad IP ranges, and provide geo-fencing. This would also mask the origin server's IP, adding another layer of defense against targeted attacks.

Recommendation #3: **Security Hardening of WordPress Backend**

Description: Limit WordPress admin panel access to whitelisted IPs through pfSense firewall rules. Enable two-factor authentication for all admin accounts. Move the wp-admin directory behind additional authentication (HTTP Auth) at the reverse proxy level to reduce exposure to brute-force attempts.

Recommendation #4: **Advanced ModSecurity Rule Tuning**

Description: Move from default OWASP CRS configuration to a tailored ModSecurity ruleset specific to MegaQuagga's traffic patterns. Disable unnecessary rules that cause false positives while adding custom rules to catch business-specific threats, such as enumeration attempts on WordPress directories.

Recommendation #5: **TLS Cipher Suite Optimization**

Description: Update SSL configuration to use only strong ciphers (e.g., TLS 1.3 and modern TLS 1.2 suites) and enable OCSP stapling for faster certificate validation. Remove legacy protocols (TLS 1.0/1.1) to prevent downgrade attacks.

Recommendation #6: **Continuous Security Testing & Incident Response Planning**

Description: Implement a rolling schedule of internal penetration tests every quarter, rotating between web application, infrastructure, and phishing/social engineering scenarios. Develop and document a formal incident response plan so that security events can be handled efficiently and consistently.



Conclusion

CONCLUSION

I implemented a reverse proxy, enabled the ModSecurity Web Application Firewall (WAF), and configured SSL offloading to secure MegaQuagga's WordPress environment. These measures remediated vulnerabilities including direct exposure to web application attacks, unfiltered malicious payload delivery, and unencrypted HTTP traffic.

Moving forward, I recommend ongoing WAF rule tuning, certificate lifecycle management, and continuous monitoring through SIEM to detect and block emerging threats.